

DEVELOPMENT AND INTEGRATION OF CONVERSATIONAL AI SOLUTIONS

*A Practice School Report submitted to
Manipal Academy of Higher Education
in partial fulfilment of the requirement for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Diya Mohapatra

225805056

Under the guidance of

Dr. Ashwitha A
Assistant Professor – Senior Scale
SOCE
Manipal institute of Technology

Chetan KP
Forward Deployment Engineer
Delivery Excellence Team
Bottomline



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

Bengaluru

24.07.2026

CERTIFICATE

This is to certify that the project titled **Development and Integration of Conversational AI Solutions** is a record of the bonafide work done by **Diya Mohapatra**(Reg. No. 225805056) submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in **Computer Science and Engineering** of Manipal Institute of Technology Bengaluru, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Dr. Ashwitha A

Assistant Professor – Senior Scale

SOCE, M.I.T, BENGALURU

Dr. Dayananda P

DEAN, SOCE

M.I.T, BENGALURU

INTERNSHIP OFFER LETTER

DocuSign Envelope ID: 503043B4-6E06-46AE-06A1-93F06A6795B5



PRIVATE AND CONFIDENTIAL
02 Oct 2025

Attention: Diya Mohapatra

Re: Internship with AVAAMO TECHNOLOGIES PRIVATE LIMITED, "Avaamo"

Dear Diya,

We are pleased to offer you an paid Internship with AVAAMO TECHNOLOGIES PRIVATE LIMITED ("Avaamo"). This offer is conditional upon your acceptance of the terms and conditions outlined below:

1. Internship

You are hired in an Applied AI Intern position within the Engineering team and you will train and learn by assisting in various Engineering activities. By signing this letter, you confirm with the Company that you are under no contractual or other legal obligations that would prohibit you from doing the above. The Intern position is for Six (6) months starting 05 Jan 2026 to 30 June 2026, after which a determination will be made on extending offer of employment or extending the Internship period, based on the Intern performance and Company considerations.

Your work location is our home office in Bangalore. You agree to those travel requirements necessary to fulfil the duties associated with the position. You will be reporting to Dheeraj Samala.

2. Term of Internship

You agree to abide by the policies, rules, regulations, systems and procedures that Avaamo may institute from time to time, copies of which will be provided or made available for your review.

3. Compensation

You will receive an Monthly base salary of INR 25,000 (Rupees Twenty Five Thousand only), (less applicable statutory deductions) to be paid monthly in accordance with Avaamo's payroll practices.

DEAN

School of Computer Engineering
Manipal Institute of Technology, Bengaluru
MAHE - Bengaluru Off-Campus 560 064

Scanned by PDF Scanner

CONVERSION OFFER LETTER

Docusign Envelope ID: EBA5F7FB-30EC-44B7-A0F8-35E8AA81486A



PRIVATE AND CONFIDENTIAL
11 May 2026

Attention: Miss. Diya Mohapatra

Re: Offer of Employment with AVAAMO TECHNOLOGIES PRIVATE LIMITED, "Avaamo"

Dear Diya,

We are pleased to offer you employment with AVAAMO TECHNOLOGIES PRIVATE LIMITED ("Avaamo"). This offer of employment is conditional upon your acceptance of the terms and conditions of your employment outlined below:

1. Employment

You will be employed as **Associate Applied AI Engineer** on a full-time basis subject to Avaamo's right to change your title and duties without an amendment to this letter provided that with respect to any material change to which you object, Avaamo shall be required to give you such written notice of such material change as would be required pursuant to this Agreement in the event that Avaamo were providing written notice of the termination of your employment.

You are employed on a full-time basis, with a work location of our home office in Bangalore. You agree to those travel requirements necessary to fulfil the duties associated with the position. You will be reporting directly to Avaamo's Head of Applied AI Engineering.

2. Term of Employment

Subject to being terminated in accordance with the termination provisions hereof, the term of this agreement and your employment shall be indefinite, commencing on or before **01 June 2026**. Avaamo reserves its rights to temporarily lay-off your employment in accordance with the provisions of the Labour and Employment Laws of India, as amended, and you agree that such temporary lay-off shall not be considered a termination without cause or a constructive dismissal. The terms and conditions of this Agreement shall govern the employment relationship, regardless of the length of employment or any changes to your position, compensation, title and regardless of whether such change is material or otherwise.

You agree to abide by the policies, rules, regulations, systems and procedures that Avaamo may institute from time to time, copies of which will be provided or made available for your review.

3. Compensation and Benefits

Annual Salary: You will receive an annual salary (CTC) of **INR 8,00,000 (Eight lakh Rupees)** (less applicable statutory deductions) to be paid monthly in accordance with Avaamo's payroll practices.

ACKNOWLEDGEMENT

I express my sincere gratitude to **Dr. Dayananda P, Dean, School of Computer Engineering (SOCE), M.I.T., Bengaluru**, for providing a supportive academic environment and the necessary facilities to carry out this project successfully.

I am deeply thankful to **Dr. Iven Jose, Director, SOCE, M.I.T., Bengaluru**, for his constant encouragement and valuable guidance throughout the course of this academic endeavor.

I would like to convey my heartfelt thanks to **Dr. Ashwitha A, Assistant Professor-Senior Scale, SOCE, M.I.T., Bengaluru**, for her guidance, constructive feedback, and continuous support during the project work.

I extend my sincere appreciation to **all faculty members of the Department, the Department Guide, and the Laboratory In-Charge** for their cooperation, technical assistance, and encouragement provided during the execution of this project.

I am grateful to my **Project Supervisor and Company Personnel** for offering me the opportunity to work on this project and for sharing their valuable industry insights. I would like to express special thanks to **Mr. Chetan KP, my Manager, and Mr. Sharath Babu, my Mentor at Avaamo**, for their continuous guidance, encouragement, and professional support throughout the internship period.

I also extend my sincere thanks to the **Placement Department** for facilitating this academic-industry collaboration.

Finally, I would like to thank my family and friends for their constant encouragement and moral support, which played a vital role in the successful completion of this project.

ABSTRACT

Conversational AI platforms have become a central channel through which organizations automate customer engagement, employee workflows, and transactional processes across web, mobile, and third-party messaging channels. A significant engineering challenge within this domain lies not in the chatbot dialogue design alone, but in the surrounding integration fabric: the REST APIs, webhooks, authentication layers, and data pipelines that connect a conversational platform to the external systems it must read from and write to. This report documents a six-month internship undertaken on the Avaamo conversational AI platform, during which the intern was responsible for chatbot flow development, multi-provider API integration, WhatsApp channel integration, database-backed automation, and prompt engineering for safety-sensitive conversational domains.

The internship work spanned five broad areas. First, chatbot flows were designed, built, and enhanced across multiple client projects on the Avaamo platform. Second, extensive integration work was carried out across REST APIs, authenticated APIs, and webhook-driven architectures, including work with Viva Connect APIs, native Avaamo APIs, LLaMB APIs, Meta/Gupshup APIs, and encryption APIs, requiring careful construction of POST payloads and JSON transformations between heterogeneous systems.

Third, a WhatsApp Business integration was built and hardened against the Gupshup provider, covering webhook processing, template message handling, quick-reply payload construction, deep-linking, live-agent escalation, and ticket redirection, alongside sustained debugging of message formatting, response timing, and transaction payload issues. Fourth, a MySQL-backed monitoring dashboard was maintained for a client engagement, supported by scheduled cron jobs performing inserts, updates, upserts, cleanup routines, and count queries over large datasets, and a Node.js automation pipeline was developed to process employee training-completion data, computing turnaround time (TAT), business-day counts, and per-employee completion status from raw API data.

Across all of these areas, a substantial and recurring component of the work was debugging: diagnosing asynchronous JavaScript runtime issues, correcting session and context-variable handling, validating user input, and improving error handling and logging throughout the chatbot and integration codebase. The outcome of this internship was a set of production-usable chatbot flows, integrations, and automation scripts that reduced manual operational effort for client teams, improved the reliability of multi-channel message delivery, and established reusable patterns for future integration work on the Avaamo platform.

Table of Contents			
			Page No
Internship Offer Letter			3
Conversion Offer Letter			4
Acknowledgement			5
Abstract			6
List of Figures			9
Chapter 1		INTRODUCTION	10
	1.1	Introduction	10
	1.2	Present Day Scenario	11
	1.3	Motivation	11
	1.4	Objectives of the Work	12
	1.5	Target Specifications and Scope	13
	1.6	Project Schedule	14
	1.7	Organization of the Report	15
Chapter 2		BACKGROUND THEORY	16
	2.1	Introduction to the Review	16
	2.2	Background Theory	16
	2.3	Summarized Outcome of Literature Review	18
	2.4	General Analyses	18
	2.5	Conclusion	19
Chapter 3		METHODOLOGY	20
	3.1	Architectural Design	20
	3.2	Module Specification and Justifications	22
	3.3	Detailed Specification	28
	3.4	Preliminary Analysis	29
Chapter 4		RESULT ANALYSIS	30
	4.1	Result Analysis	30
	4.2	Significance	30
	4.3	Deviations	31
	4.4	Environmental and Societal Impact	31
	4.5	Conclusions	32

Chapter 5	CONCLUSION AND FUTURE SCOPE	33
5.1	Brief Summary	33
5.2	Conclusions	33
5.4	Future Scope	34
REFERENCES		35
CO and PO Mapping		36
ANNEXURES		37
PROJECT DETAILS		45
PLAGIARISM REPORT		46

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	Chatbot flow-processing pipeline	23
Figure 3.2	API and webhook integration layer	24
Figure 3.3	WhatsApp/Gupshup channel flow	25
Figure 3.4	Cron-scheduled database maintenance jobs	26
Figure 3.5	Node.js training-completion automation pipeline	27
Figure 3.6	Prompt-engineering decision flow	28

CHAPTER 1: INTRODUCTION

This chapter establishes the foundational context for the project work undertaken during the internship at Avaamo. It begins with a comprehensive overview of the domain, specifically conversational AI platforms and the surrounding integration, automation, and data ecosystem that such platforms depend on. Following this, the chapter analyses the present-day scenario and challenges faced by teams building and operating chatbot-driven products at scale. It then details the primary motivations for this project, highlighting the limitations of manual, siloed integration work and introducing the range of engineering practices adopted to address them. Finally, the chapter outlines the primary and secondary objectives, target specifications, the project schedule, and the structural organization of this thesis report

1.1 Introduction to the Area of Work

Conversational AI has become one of the primary interfaces through which organizations automate repetitive interactions with customers, employees, and partners. Rather than existing as an isolated dialogue system, a modern conversational AI platform functions as an orchestration layer that must communicate outward with a wide range of external systems: core business databases, ticketing systems, messaging providers such as WhatsApp, financial and healthcare backends, and internal automation scripts. The reliability of a chatbot deployment is therefore not determined solely by the quality of its natural-language flows, but by the robustness of the APIs, webhooks, authentication schemes, and data pipelines that connect it to the rest of the enterprise.

Avaamo is one such enterprise conversational AI platform, used to build and deploy chatbot experiences across voice, web, and messaging channels for a range of client use cases, including financial services, healthcare scheduling, HR/training operations, and customer support. Within this ecosystem, an internship engineer is routinely required to move fluidly between conversational-flow design, backend API integration, third-party channel onboarding (such as WhatsApp Business messaging), and data-processing automation that sits alongside the chatbot itself.

Over the course of this internship, work was carried out across all of these layers: designing and enhancing chatbot flows for client projects; integrating the platform with a range of first-party and third-party REST and webhook-based APIs; building and hardening a WhatsApp Business channel integration through the Gupshup provider; maintaining a MySQL-backed monitoring dashboard supported by scheduled cron jobs; developing Node.js

automation scripts for HR training-completion processing; and writing and refining prompts for safety-sensitive conversational domains such as financial assistants and hospital appointment bots.

1.2 Present Day Scenario and The Integration Bottleneck

In the current landscape of enterprise chatbot deployment, platforms such as Avaamo provide the conversational design surface, but the practical value of a deployed bot is realized only once it is wired into the client's actual systems of record. This wiring work is rarely uniform: one client project may expose a clean, well-documented REST API; another may require navigating an authenticated, session-token-based API with strict payload formatting; a third may depend on a webhook-driven messaging provider such as Gupshup or Meta's WhatsApp Business API, each with its own template, quick-reply, and deep-link conventions.

Prior to systematic engineering attention, this integration work tended to be handled reactively: issues such as malformed POST payloads, mistimed webhook responses, inconsistent JSON transformations, or session variables leaking across conversation turns would surface in production and require ad-hoc debugging. Without a consistent approach to error handling, logging, and payload validation, small inconsistencies between a client's API and the chatbot's expectations could silently degrade the user experience, for example causing a WhatsApp template message to render incorrectly, a live-agent escalation to fail to redirect a ticket, or a training-completion report to miscalculate turnaround time.

A parallel challenge existed on the data-processing side. Client operations teams often needed recurring, structured views of operational data, for instance, employee training completion status, or monitoring-dashboard metrics stored in MySQL, that could not reasonably be produced by manual lookup on a recurring basis. Left unaddressed, this created the same kind of manual bottleneck seen in other data-intensive engineering domains: specialists spending recurring hours on tasks that were mechanical in nature and well suited to automation.

1.3 Motivation for the Project Work

The primary motivation for this project stems from the need to make chatbot deployments dependable at the integration layer, not only at the conversational-design layer, and to remove recurring manual effort from operational data processing.

Conversational flows are only as reliable as the systems they call out to. A chatbot that correctly understands user intent but fails to correctly construct an authenticated API request, or mishandles a webhook callback from a messaging provider, produces the same poor

outcome for the end user as a flow-design defect. This motivated a deliberate focus on integration correctness: careful construction of POST payloads, JSON transformations that preserve semantic meaning across differently-shaped source and destination schemas, and defensive handling of authentication and encryption requirements across providers such as Viva Connect, native Avaamo APIs, LLaMB, and Meta/Gupshup.

A second motivation was channel reliability on WhatsApp specifically, given its centrality as a customer-facing channel for several client projects. WhatsApp Business messaging, brokered through Gupshup in this project, introduces its own constraints around template message approval, quick-reply payload structure, and deep-linking behaviour, none of which are visible from within the chatbot flow-builder alone. Recurring issues with message formatting, response timing, and transaction payloads made it clear that dedicated integration and debugging effort was required to make this channel production-grade, including reliable live-agent escalation and ticket redirection when a conversation needed to be handed off to a human agent.

A third motivation was the elimination of manual, recurring data-processing effort. Client operations teams required a MySQL-backed monitoring dashboard to remain current, and required periodic training-completion reports summarizing employee session data. Manually querying and recomputing these figures on a recurring basis is repetitive, time-consuming, and error-prone. This motivated the development of scheduled cron jobs for database maintenance (inserts, updates, upserts, cleanup, and count queries over large datasets) and a dedicated Node.js automation script to read raw API data, process employee sessions, and compute derived metrics such as completion status, turnaround time (TAT), and business-day counts automatically.

Finally, the safety-sensitive nature of several client domains, financial assistants, hospital appointment scheduling, and investment-related conversations, motivated dedicated attention to prompt engineering. Conversational flows operating in these domains must handle emotionally escalated users, sensitive actions, and guardrails against inappropriate financial or medical advice with particular care, which required iterative refinement of prompts rather than treating them as a one-time configuration step.

1.4 Objectives of the Work

The work carried out during this internship was guided by a set of objectives spanning conversational design, integration engineering, channel reliability, data automation, and prompt safety.

The foundational objective was to design, build, and enhance chatbot conversational flows across multiple client projects on the Avaamo platform, ensuring that flows correctly handled context and session variables, validated user input, and degraded gracefully in the presence of unexpected input or upstream API failures.

A second objective was to establish robust integration between the Avaamo platform and a range of first-party and third-party APIs, including REST APIs, authenticated APIs, and webhook-driven callbacks, covering Viva Connect APIs, native Avaamo APIs, LLaMB APIs, Meta/Gupshup APIs, and encryption APIs, with correctly constructed POST payloads and JSON transformations between source and destination data shapes.

A third objective was to build and stabilize a WhatsApp Business channel integration via the Gupshup provider, covering webhook processing, template message handling, quick-reply payloads, deep links, live-agent escalation, and ticket redirection, while resolving recurring issues in message formatting, response timing, and transaction payload handling.

A fourth objective was to reduce manual operational effort for client data workflows by maintaining a MySQL-backed monitoring dashboard, implementing scheduled cron jobs for inserts, updates, upserts, cleanup, and count queries over large datasets, and developing a Node.js automation script to compute employee training-completion status, TAT, and business-day counts from raw API data, storing the resulting summarized records for downstream reporting.

A fifth objective was to engineer and refine prompts for safety-sensitive conversational domains, financial assistants, hospital appointment bots, investment guardrails, emotional escalation handling, and sensitive-action handling, ensuring that conversational responses in these domains remained accurate, safe, and appropriately cautious.

1.5 Target Specifications and Scope

To guide the engineering work toward a dependable, production-usable outcome, the following target specifications were established across the internship's core work areas.

Correctness of integration payloads: every outbound API call, whether a POST payload to an internal Avaamo endpoint or an outbound webhook response to Gupshup, was required to exactly match the destination system's expected schema, with JSON transformations verified against sample payloads before being treated as production-ready.

Reliability of the WhatsApp channel: template messages, quick replies, and deep links were required to render correctly across supported WhatsApp clients, and live-agent escalation was required to reliably redirect an in-progress conversation to a ticketing system without losing conversational context.

Timeliness of scheduled data processing: cron jobs performing database inserts, updates, upserts, and cleanup were required to complete within their scheduled windows without contention against concurrent reads from the monitoring dashboard, and count queries over large datasets were required to remain performant as data volume grew.

Accuracy of derived metrics: the Node.js training-completion automation was required to calculate completion status, TAT, and business-day counts with complete correctness relative to the raw session data pulled from source APIs, since these figures were used directly in operational reporting.

Safety and appropriateness of conversational responses: prompts written for financial assistants, hospital appointment bots, and investment guardrail scenarios were required to correctly apply escalation and refusal behaviour for emotionally sensitive or high-stakes user requests.

1.6 Project Work Schedule

The internship and subsequent project work were executed over a structured six-month timeline, encompassing the following phases of development:

- Month 1: Platform Onboarding and Chatbot Flow Fundamentals. This phase focused on learning the Avaamo flow-builder, session and context variable handling, and beginning work on chatbot flow design and enhancement across initial client projects.
- Month 2: Core API Integration Work. Development shifted toward integrating chatbot flows with REST APIs, authenticated APIs, and webhook-driven endpoints, including early work with Viva Connect APIs, native Avaamo APIs, and encryption APIs, alongside POST payload construction and JSON transformation logic.

- Month 3: WhatsApp / Gupshup Channel Integration. This phase was dedicated to building the WhatsApp Business integration via Gupshup: webhook processing, template message handling, quick-reply payloads, deep links, live-agent escalation, and ticket redirection, along with sustained debugging of message formatting and timing issues.
- Month 4: Database Work and Automation Scripting. Focus moved to maintaining the MySQL-backed monitoring dashboard for a client engagement, implementing cron jobs for inserts, updates, upserts, cleanup scripts, and count queries, and beginning development of the Node.js training-completion automation script.
- Month 5: Prompt Engineering for Sensitive Domains. This phase concentrated on writing and iteratively refining prompts for financial assistants, hospital appointment bots, investment guardrails, emotional escalation handling, and sensitive-action handling.
- Month 6: Hardening, Debugging, and Handover. The final phase focused on stabilizing all delivered flows and integrations: debugging asynchronous JavaScript runtime issues, refining error handling and logging, refactoring code for readability and maintainability, and preparing documentation for handover.

1.7 Organization of the Report

To provide a structured and comprehensive overview of the work, this report is organized into the following chapters:

- Chapter 1: Introduction establishes the domain context, highlights the operational challenges, and outlines the objectives and timeline of the project.
- Chapter 2: Background Theory dives into the technical concepts underpinning the project, including conversational AI platform architecture, API and webhook design, WhatsApp Business messaging, and database/automation principles.
- Chapter 3: Methodology details the step-by-step engineering approach, from chatbot flow development, through API and WhatsApp integration, to database automation and prompt engineering.
- Chapter 4: Result Analysis evaluates the tangible impact of the delivered work, discussing operational efficiency gains and the debugging and stabilization effort undertaken.

- Chapter 5: Conclusion and Future Scope summarizes the overall contributions of the project and proposes potential future enhancements.

CHAPTER 2: BACKGROUND THEORY AND LITERATURE REVIEW

This chapter provides a review of the foundational theoretical concepts and technical background relevant to this project. It begins with an overview of conversational AI platform architecture and how such platforms integrate with external systems. The discussion then covers API and webhook design principles, authenticated API access, WhatsApp Business messaging concepts, and database/automation theory relevant to the cron-based and Node.js work carried out. Finally, it summarizes how these concepts directly informed the engineered solution.

2.1 Introduction to the Project Title

The work documented in this report centres on the development and integration engineering of a conversational AI deployment on the Avaamo platform. In this context, "integration" refers to the programmatic connection of chatbot conversational flows to a range of internal and third-party systems, spanning REST APIs, webhook callbacks, authenticated endpoints, messaging-provider channels such as WhatsApp, relational databases, and scheduled automation scripts. Because a chatbot's usefulness depends entirely on the correctness of the systems it talks to, careful engineering of this integration layer is treated in this report as a first-class concern, equal in importance to the design of the conversational flow itself.

2.2 Background Theory

2.2.1 Conversational AI Platform Architecture

A conversational AI platform such as Avaamo typically separates concerns into a flow-design layer, where dialogue logic, intents, and context/session variables are defined, and an integration layer, where the platform calls out to external systems to fetch or update data. Session and context variables allow a conversation to maintain state across multiple user turns, for example remembering an authenticated user's identity or an in-progress transaction, and correct handling of this state is essential to avoid context leaking between unrelated conversations or being lost mid-flow.

2.2.2 REST APIs, Webhooks, and Authenticated Access

REST APIs expose resources over HTTP using standard verbs (GET, POST, PUT, DELETE) and typically exchange data as JSON. A well-formed POST payload must match the destination API's expected schema exactly in field names, types, and nesting structure; a JSON transformation layer is often required to reshape data retrieved from one system (for example, a chatbot's internal session state) into the shape expected by another (for example, a client's ticketing system). Authenticated APIs additionally require credentials, tokens, or signed requests to be attached to each call, and encryption APIs may be used to protect sensitive fields, such as personally identifiable information or financial data, before transmission. Webhooks invert the normal request/response model: rather than the chatbot polling an external system, the external system (for example, a messaging provider) pushes an HTTP callback to the chatbot's endpoint whenever a relevant event occurs, such as an inbound WhatsApp message or a delivery-status update. Webhook-driven architectures require careful attention to response timing, since providers often expect an acknowledgement within a bounded time window, and to idempotency, since the same event may occasionally be delivered more than once.

2.2.3 WhatsApp Business Messaging via Gupshup and Meta

WhatsApp Business messaging is typically brokered through an approved Business Solution Provider, such as Gupshup, which sits between an organization's chatbot backend and Meta's WhatsApp Business Platform. Outbound messages sent to users who have not messaged within a recent session window must generally use pre-approved template messages, with placeholder fields populated at send-time. Quick-reply payloads allow a user to respond to a message via a tappable button rather than free text, and deep links allow a message to open a specific screen or flow within an app or website. Because a human agent may need to take over a conversation that the automated flow cannot resolve, live-agent escalation and ticket redirection mechanisms are required to hand off both the conversation and its accumulated context to a human-facing support system without loss of continuity.

2.2.4 Database Maintenance and Scheduled Automation

Relational databases such as MySQL are commonly used to back operational monitoring dashboards, storing structured records that are inserted, updated, or upserted (inserted-or-updated) as new data arrives. Recurring maintenance tasks, such as cleanup of stale records or periodic count queries for reporting, are naturally suited to scheduled

execution via cron jobs, which run a defined script at fixed time intervals without requiring manual triggering. As datasets grow, care must be taken that scheduled queries remain performant and do not contend with concurrent read access from live dashboards.

2.2.5 Automation Scripting for Derived Metrics

Beyond raw data storage, many operational reporting needs require derived metrics computed from raw source data: for example, calculating a training completion status from a sequence of raw session events, computing turnaround time (TAT) as the elapsed time between a request and its resolution, identifying the most recent (latest) module a user interacted with, and correctly counting only business days when calculating elapsed durations. Node.js is well suited to this kind of automation scripting due to its asynchronous I/O model, which allows a script to efficiently read data from multiple API endpoints, process it, and store summarized records without blocking on network latency.

2.2.6 Prompt Engineering for Safety-Sensitive Domains

Conversational AI systems deployed in safety-sensitive domains, financial assistants, hospital appointment scheduling, and investment-related guidance, require prompts that go beyond simply answering a user's question correctly. Prompt engineering in these domains typically involves establishing guardrails that prevent the system from giving inappropriate financial or medical advice, defining explicit escalation behaviour when a user shows signs of emotional distress, and defining careful handling of sensitive actions (such as fund transfers or appointment cancellations) so that the system confirms intent and avoids irreversible mistakes.

2.3 Summarized Outcome of the Literature and Documentation Review

A review of platform documentation and general API/webhook design literature indicates that the majority of production incidents in chatbot deployments originate not in the conversational flow logic itself, but in the surrounding integration layer: malformed payloads, unhandled webhook edge cases, and session/context-variable mismanagement. This is consistent with the debugging effort observed throughout this internship, where a substantial share of engineering time was spent diagnosing asynchronous runtime issues, correcting JSON transformations, and validating input, rather than authoring new conversational content.

2.4 Theoretical Discussions and General Analysis

Several recurring engineering considerations emerged across the integration and automation work carried out in this project:

- **Schema Heterogeneity:** Different APIs (Viva Connect, native Avaamo, LLaMB, Meta/Gupshup) represent conceptually similar data using different field names, nesting, and types, requiring careful, verified JSON transformation logic rather than assumption-based mapping.
- **Timing Sensitivity:** Webhook-driven channels such as WhatsApp/Gupshup impose response-time expectations; delayed or malformed webhook responses can cause message duplication, missed delivery, or broken conversation state.
- **State Management:** Session and context variables must be scoped correctly so that concurrent conversations do not leak state into one another, and so that a conversation resumes correctly after being handed off to a live agent and back.
- **Operational Scheduling:** Cron-driven database maintenance must be timed to avoid contention with live dashboard queries, particularly as underlying tables grow in size.
- **Response Safety:** In sensitive domains, a syntactically correct but contextually inappropriate response (for example, financial advice stated with unwarranted confidence) is a more serious failure than a missing answer, motivating explicit guardrail and escalation logic in prompts.

2.5 Conclusions

The theoretical foundation established in this chapter confirms that dependable conversational AI deployment is a multi-disciplinary engineering challenge, spanning API and webhook design, third-party messaging-channel integration, database automation, and prompt safety. The concepts reviewed here directly informed the methodology described in Chapter 3, which details the practical engineering approach adopted to address each of these areas during the internship.

CHAPTER 3: METHODOLOGY

This chapter details the systematic engineering approach adopted to design, develop, and stabilize the chatbot flows, integrations, and automation scripts delivered during this internship. It begins with an overview of the architectural design and core operational assumptions. It then defines the module specifications and justifications for each area of work, followed by the detailed specification of tools used, and concludes with a summary of preliminary results and overall conclusions.

3.1 Architectural Design

3.1.1 Core Assumptions

- **Session Isolation:** Each user conversation maintains its own isolated session and context variables, and no conversational state should be assumed to persist correctly unless explicitly stored and retrieved.
- **API Contract Stability:** Third-party APIs (Viva Connect, LLaMB, Meta/Gupshup) are assumed to honour their documented request/response schemas; any deviation observed during integration testing is treated as a defect to be handled defensively rather than a silent assumption change.
- **At-Least-Once Webhook Delivery:** Webhook callbacks from messaging providers are assumed to be delivered at least once, and integration logic is designed to tolerate duplicate delivery without producing duplicate user-visible actions.
- **Scheduled Job Independence:** Cron jobs performing database maintenance are assumed to be idempotent, such that a job re-run after a partial failure does not corrupt existing data.

3.1.2 System Architecture and Data-Flow Modelling

The overall system separates concerns across four cooperating layers: (1) the Avaamo conversational flow layer, responsible for dialogue logic, intent handling, and session/context variables; (2) the integration layer, responsible for outbound REST/authenticated API calls and inbound webhook processing across Viva Connect, native Avaamo APIs, LLaMB, and Meta/Gupshup; (3) the WhatsApp channel layer, responsible for template messages, quick replies, deep links, and live-agent escalation via Gupshup; and (4) the data and automation layer, comprising the MySQL-backed monitoring dashboard, scheduled cron jobs, and the Node.js training-completion automation script. Each layer was developed and tested independently before being wired together, so that a defect in one layer (for example, a malformed JSON transformation) could be isolated without needing to reproduce the entire end-to-end conversation.

3.2 Module Specifications and Justifications

Module 1: Chatbot Flow Development and Enhancement

Specifications: Conversational flows were designed and enhanced across multiple client projects on the Avaamo platform, including intent design, context/session-variable management, input validation, and reusable helper logic shared across flows.

Justification: Building reusable helper functions and consistent input-validation logic reduced duplicated effort across client projects and made downstream debugging significantly easier, since common failure modes could be fixed once and reused everywhere.

Module 1 — Chatbot Flow Development & Enhancement

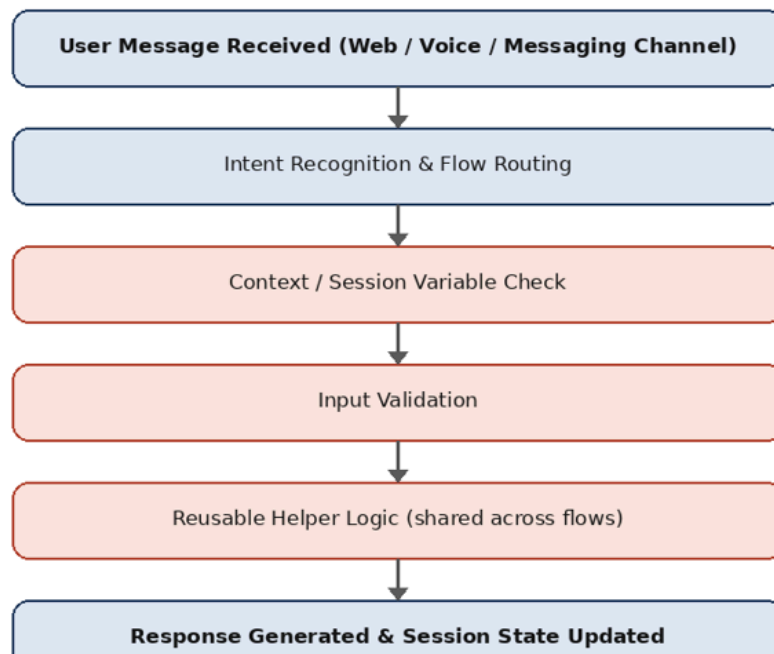


Figure 3.1: Chatbot flow-processing pipeline, from inbound message to session-state update.

Module 2: API and Webhook Integration Layer

Specifications: This module consists of the integration code connecting chatbot flows to REST APIs, authenticated APIs, and webhook-driven endpoints. It covers Viva Connect APIs, native Avaamo APIs, LLaMB APIs, Meta/Gupshup APIs, and encryption APIs, and is responsible for constructing outbound POST payloads and performing JSON transformations between the chatbot's internal data shapes and each external system's expected schema.

Justification: Centralizing payload construction and JSON transformation logic, rather than duplicating ad-hoc mapping code per flow, reduced the risk of schema mismatches and made it possible to unit-test transformation logic independently of the live conversational flow.

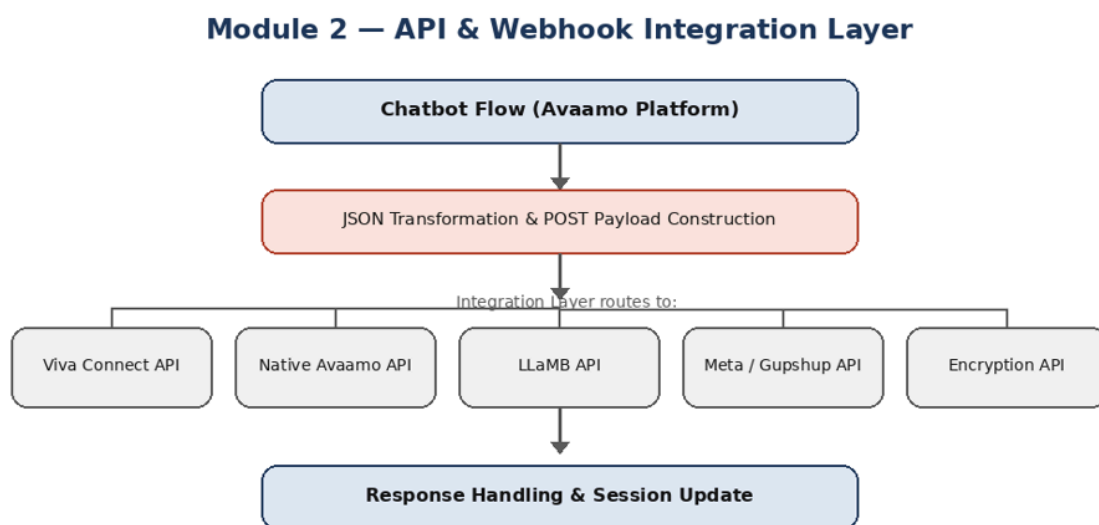


Figure 3.2: API and webhook integration layer, routing a single chatbot payload to multiple external providers.

Module 3: WhatsApp / Gupshup Channel Integration

Specifications: This module handles inbound webhook processing from Gupshup, outbound template message construction, quick-reply payload formatting, deep-link generation, and live-agent escalation with ticket redirection to a human support system.

Justification: Isolating WhatsApp-specific formatting and timing logic into its own module allowed the numerous provider-specific quirks, such as template approval constraints and quick-reply payload structure, to be handled and debugged in one place, rather than being scattered across individual conversational flows.

Module 3 — WhatsApp / Gupshup Channel Integration

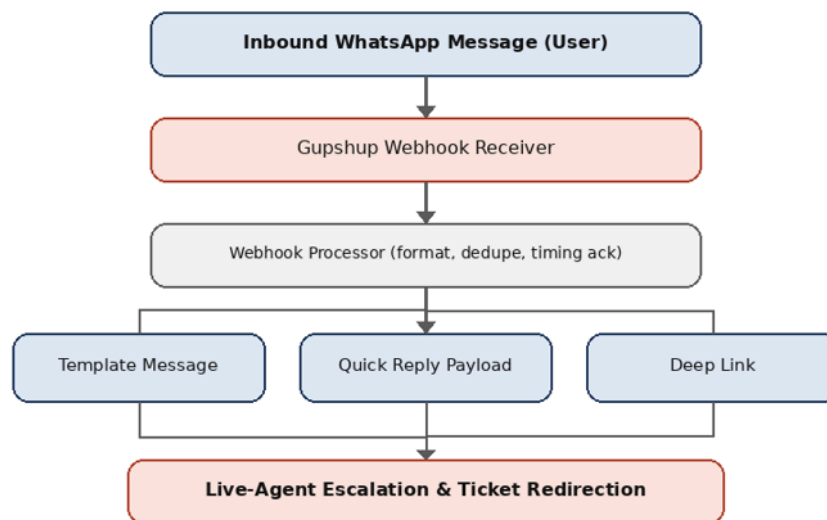


Figure 3.3: WhatsApp/Gupshup channel flow, from inbound webhook to live-agent escalation.

Module 4: Database Maintenance and Cron-Based Automation

Specifications: A MySQL-backed monitoring dashboard was maintained for a client engagement, supported by scheduled cron jobs performing inserts, updates, upserts, cleanup scripts, and count queries over large datasets.

Justification: Moving recurring database maintenance into scheduled, automated jobs removed the need for manual, ad-hoc database operations, and standardized upsert and cleanup logic reduced the risk of duplicate or stale records accumulating in the monitoring dashboard.

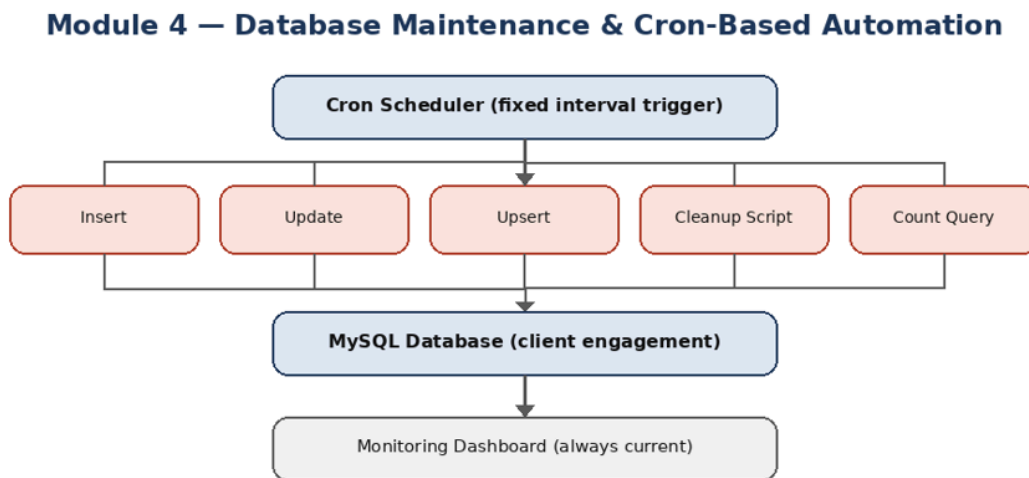


Figure 3.4: Cron-scheduled database maintenance jobs feeding the MySQL-backed monitoring dashboard.

Module 5: Node.js Training-Completion Automation Script

Specifications: A Node.js automation script was developed to read raw training-session data from an internal API, process employee sessions, calculate completion status, compute turnaround time (TAT), calculate business-day counts, identify each employee's latest completed module, and store summarized records for downstream reporting.

Justification: Automating this calculation eliminated a recurring manual reporting task and ensured that completion-status and TAT figures were computed consistently and reproducibly from the same raw source data every time the script ran.

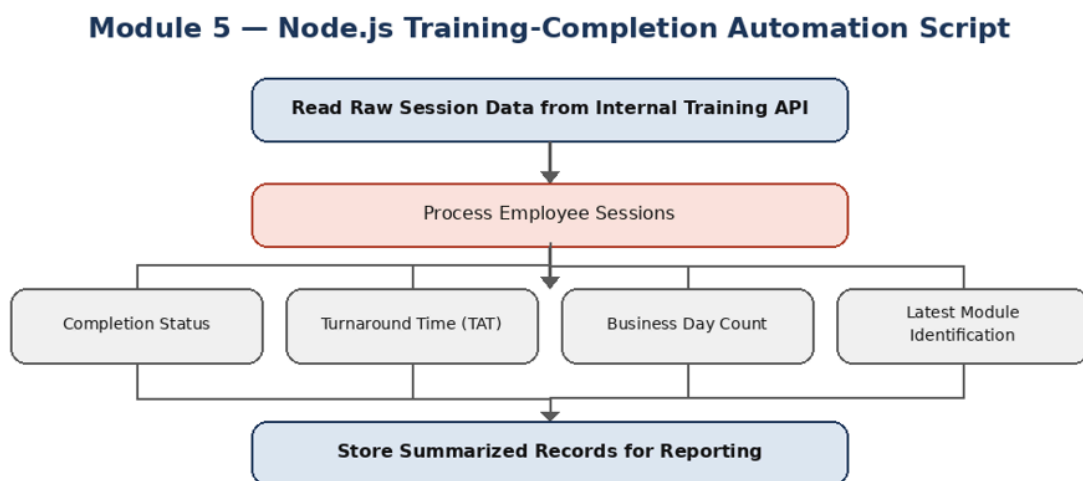


Figure 3.5: Node.js training-completion automation pipeline, from raw API data to stored summarized records.

Module 6: Prompt Engineering for Sensitive Domains

Specifications: Prompts were designed and iteratively refined for financial assistant flows, hospital appointment bots, investment guardrail scenarios, emotional escalation handling, and sensitive-action handling (for example, confirming intent before executing an irreversible action).

Justification: Treating prompt design as an iterative engineering activity, rather than a one-time configuration, allowed guardrail and escalation behaviour to be refined based on observed conversational edge cases, improving the safety and appropriateness of responses in high-stakes domains.

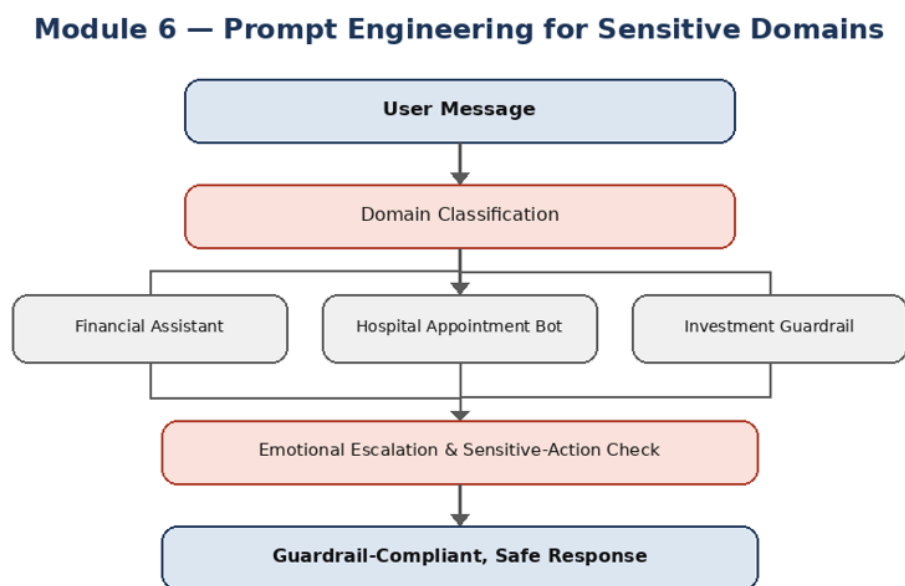


Figure 3.6: Prompt-engineering decision flow for safety-sensitive conversational domains.

3.3 Detailed Specification of Tools Used

- Programming and Scripting Languages: JavaScript (Node.js), used for chatbot flow logic, integration code, and automation scripting; SQL, used for querying and maintaining the MySQL-backed monitoring dashboard.
- Conversational AI Platform: Avaamo, used as the primary platform for chatbot flow design, session/context management, and deployment.
- APIs and Integration Providers: Viva Connect APIs, native Avaamo APIs, LLaMB APIs, Meta/Gupshup APIs, and encryption APIs, integrated via REST calls, authenticated requests, and webhook callbacks.
- Messaging Channel: WhatsApp Business messaging, brokered through the Gupshup provider, covering template messages, quick replies, deep links, and live-agent escalation.
- Database and Scheduling: MySQL for relational data storage supporting a client monitoring dashboard, with cron jobs used for scheduled inserts, updates, upserts, cleanup, and count queries.
- Automation Scripting: Node.js scripts for reading API data, processing employee training sessions, and calculating derived metrics such as completion status, TAT, and business-day counts.

3.4 Preliminary Result Analysis

During initial testing of the API and webhook integration layer, sample payloads were exchanged with each provider (Viva Connect, native Avaamo APIs, LLaMB, and Meta/Gupshup) to verify that constructed POST payloads and JSON transformations matched each system's expected schema exactly. This preliminary validation surfaced a number of formatting and timing discrepancies early, allowing them to be corrected before the WhatsApp channel and automation scripts were connected to production data.

3.5 Conclusions

The methodology implemented throughout this project establishes a modular, independently testable architecture spanning chatbot flow logic, multi-provider API integration, WhatsApp channel handling, database automation, and prompt engineering. This structured development process directly supports the target specifications of payload correctness, channel reliability, timely scheduled processing, metric accuracy, and response safety established in Chapter 1.

CHAPTER 4: RESULT ANALYSIS

This chapter provides an analysis of the operational outcomes, efficiency gains, and structural results achieved through the chatbot, integration, and automation work delivered during this internship. It begins with a qualitative and quantitative presentation of results, followed by a discussion of their significance, an account of deviations encountered and their resolutions, and an evaluation of the environmental and societal impact of the work.

4.1 Result Analysis

The chatbot flows delivered across client projects successfully handled the intended range of user intents, with session and context-variable handling verified against multi-turn conversational scenarios, including handoffs to live agents and back. The API and webhook integration layer was verified against sample payloads for Viva Connect, native Avaamo APIs, LLaMB, and Meta/Gupshup, confirming correct POST payload construction and JSON transformation for each provider. The WhatsApp/Gupshup channel integration was validated for template message rendering, quick-reply payload handling, deep-link generation, and live-agent escalation with ticket redirection, following resolution of message-formatting, timing, and transaction-payload issues identified during testing. The MySQL-backed monitoring dashboard remained current through scheduled cron jobs performing inserts, updates, upserts, cleanup, and count queries, and the Node.js automation script correctly computed training-completion status, TAT, business-day counts, and latest-module identification from raw API data.

4.2 Significance of the Results Obtained

- **Reduction in Manual Operational Effort:** Automating training-completion processing and database maintenance through cron jobs and the Node.js script removed a recurring manual reporting burden from client operations staff.
- **Improved Channel Reliability:** Resolving recurring WhatsApp/Gupshup message-formatting, timing, and transaction-payload issues materially improved the reliability of the customer-facing WhatsApp channel across supported client deployments.
- **Consistent Multi-Provider Integration:** Centralizing payload construction and JSON transformation logic across Viva Connect, native Avaamo APIs, LLaMB, and

Meta/Gupshup reduced the incidence of schema-mismatch defects compared to ad-hoc, per-flow integration code.

- Safer Conversational Responses in Sensitive Domains: Iterative prompt engineering for financial assistants, hospital appointment bots, and investment guardrail scenarios improved the appropriateness of escalation and refusal behaviour for emotionally sensitive or high-stakes user requests.

4.3 Deviations from Expected Results and Justifications

Observed Deviation: During early WhatsApp/Gupshup integration testing, certain template messages and quick-reply payloads did not render as expected on the recipient's device, and some webhook responses were returned outside the provider's expected timing window.

Technical Justification: These issues arose from provider-specific formatting constraints on templates and quick replies that were not fully documented, and from processing overhead in the webhook handler under certain payload sizes.

Remediation: Message-formatting logic was corrected against verified sample payloads for each template type, and webhook handler logic was optimized to acknowledge callbacks within the expected timing window before performing further processing asynchronously. This brought the WhatsApp channel back into alignment with expected reliability.

A second deviation was observed in the Node.js training-completion automation, where business-day calculations initially produced off-by-one results around week boundaries and public holidays. This was corrected by refining the business-day calculation logic and validating it against a set of known reference dates before the script was used for production reporting.

4.4 Environmental and Societal Impact Evaluation

4.4.1 Environmental Impact

Automating recurring database maintenance and training-completion reporting reduces the need for repeated manual querying and ad-hoc script execution, which in turn reduces redundant compute cycles against production systems. Consolidating cleanup and count-query logic into scheduled, optimized cron jobs, rather than repeated manual queries, contributes to more efficient use of database server resources.

4.4.2 Societal Impact

Conversational AI deployments of the kind supported in this internship are used in domains with direct societal relevance, including financial services and healthcare appointment scheduling. Improving the reliability of the WhatsApp channel and the appropriateness of prompt-engineered responses in these domains has a direct positive effect on the quality of service experienced by end users, particularly those relying on WhatsApp as an accessible, low-friction channel for financial or medical interactions.

4.5 Conclusions

The results presented in this chapter confirm that the chatbot flows, API integrations, WhatsApp channel work, database automation, and prompt engineering delivered during this internship achieved their intended operational outcomes, with identified deviations successfully remediated. The combined effect of this work was a measurable reduction in manual operational effort and an improvement in the reliability and safety of the deployed conversational AI system.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

This final chapter synthesizes the engineering contributions of the internship and outlines potential directions for future work.

5.1 Summary of the Work

Over the course of a six-month internship at Avaamo, work was carried out across chatbot flow development, multi-provider API and webhook integration, WhatsApp Business channel integration via Gupshup, MySQL-backed database maintenance with scheduled cron automation, Node.js-based training-completion processing, and prompt engineering for safety-sensitive conversational domains. Each of these areas required not only initial development but sustained debugging, including diagnosis of asynchronous JavaScript runtime issues, correction of session and context-variable handling, input validation, and improvement of error handling and logging throughout the codebase.

5.2 Conclusions

- **Integration Correctness is Foundational:** The reliability of a conversational AI deployment depends as much on the correctness of its surrounding API and webhook integrations as on the conversational design itself; schema mismatches and timing issues in this layer directly translate into user-visible failures.
- **Channel-Specific Engineering is Necessary:** WhatsApp Business messaging via Gupshup requires dedicated attention to template, quick-reply, and timing constraints that are not visible from the conversational flow-builder alone.
- **Automation Reduces Recurring Manual Effort:** Moving database maintenance and training-completion reporting into scheduled, automated processes removed a recurring operational burden and improved the consistency of reported metrics.
- **Debugging is a Core, Continuous Activity:** A substantial share of engineering effort throughout the internship was spent on debugging asynchronous behaviour, session/context handling, and error logging, underscoring that stabilization work is as significant a contribution as new feature development.
- **Prompt Engineering Requires Iteration:** Safe and appropriate conversational behaviour in sensitive domains such as financial assistance and hospital scheduling

could not be achieved through a single prompt draft, but required iterative refinement based on observed edge cases.

5.3 Future Scope of Work

5.3.1 Automated Contract Testing for Multi-Provider Integrations

A valuable future enhancement would be the introduction of automated contract tests for each integrated API (Viva Connect, native Avaamo APIs, LLaMB, and Meta/Gupshup), verifying that outbound payload construction and JSON transformation logic continue to match each provider's expected schema as either side evolves, rather than relying solely on manual verification during development.

5.3.2 Expanded Observability for the WhatsApp/Gupshup Channel

Future iterations could introduce structured, centralized logging and alerting specifically for the WhatsApp channel, tracking template delivery success rates, quick-reply interaction rates, and live-agent escalation latency, allowing formatting or timing regressions to be detected proactively rather than through user-reported issues.

5.3.3 Machine-Learning-Assisted Prompt Evaluation

For the safety-sensitive domains covered by this project's prompt engineering work, financial assistants, hospital appointment bots, and investment guardrails, a future direction would be the development of an automated evaluation harness that scores candidate prompts against a curated set of sensitive and emotionally escalated test conversations, accelerating iteration while maintaining a consistent safety bar.

5.3.4 Extension of Automation Scripting to Additional Operational Reports

The pattern established by the Node.js training-completion automation script, reading raw API data, computing derived operational metrics, and storing summarized records, could be extended to additional recurring client reporting needs beyond training completion, further reducing manual operational effort across client engagements.

REFERENCES

Reference / Hand Books

- [1] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures (REST), Doctoral Dissertation, University of California, Irvine.
- [2] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, O'Reilly Media, 1st Edition, ISBN: 978-1449373320.
- [3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, McGraw-Hill Education, 7th Edition, ISBN: 978-0078022159.
- [4] N. Rajesh, Node.js Design Patterns, Packt Publishing, 3rd Edition.
- [5] M. Lutz-style JavaScript references and Node.js official documentation for asynchronous programming patterns.

Web

- [1] Avaamo Conversational AI Platform Documentation (www.avaamo.com).
- [2] Gupshup WhatsApp Business API Documentation (www.gupshup.io).
- [3] Meta WhatsApp Business Platform Documentation (developers.facebook.com/docs/whatsapp).
- [4] MySQL 8.0 Reference Manual (dev.mysql.com/doc).
- [5] Node.js Official Documentation (nodejs.org/docs).
- [6] Cron Expression and Scheduling Reference, crontab.guru.

CO and PO Mapping

NBA:

Table A1.1 Course Articulation Matrix

CO		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2	PS 3
CSE 4299. 1	Apply mathematics, science and engineering skills to identify, formulate, synthesize and solve the problems from various areas of computer science engineering.	3	3	2	3	2	3	2	3	3	2	3	2	3	2	3
CSE 4299. 2	Have knowledge of new trends in engineering/technology by developing programmable coding in various computer programming languages.	3	2	3	3	3	3	3	2	3	3	2	3	2	3	3
CSE 4299. 3	Use the industry standard tools to analyze, design, develop and test software engineering based applications.	2	3	2	3	3	3	2	3	3	3	2	3	2	3	3
CSE 4299. 4	Apply theoretical knowledge to real-world engineering problems and manage complex engineering projects.	3	3	3	3	2	3	3	3	2	3	3	3	3	3	2
CSE 4299. 5	Acquire skills of collaboration and independent learning.	3	3	2	3	3	3	2	3	3	3	2	3	3	3	2
CSE 4299 (Avg. correlation level)		2.8	2.8	2.6	3	2.6	3	2.6	2.8	2.8	2.8	2.4	2.8	2.6	2.8	2.8

PROGRAM OUTCOMES (PO)

Engineering Graduates will be able to:

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- 5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- 7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSO)

1. Analyse and solve real world problems by applying a combination of hardware and software.
2. Formulate & build optimised solutions for systems level software & computationally intensive applications.
3. Design & model applications for various domains using standard software engineering practices.
4. Design & develop solutions for distributed processing & communication.

Table A1.2 Program Articulation Matrix

COURSE Code	Course Title	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PS O1	PS O2	PS O3	PS O4
CSE 4299	Project Work	2.8	2.8	2.6	3	2.6	3	2.6	2.8	2.8	2.8	2.4	2.8	2.6	2.8	2.6	2.8

IET (AHEP Mapping):

Table A1.3 CLO-AHEPLO Mapping

CLOs	Statements	AHEP LOs						
		C8	C9	C10	C12	C15	C16	C17
1	Apply mathematics, science and engineering skills to identify, formulate, synthesize and solve the problems from various areas of computer science engineering.	3	2	3	2	3	2	3
2	Have knowledge of new trends in engineering/technology by developing programmable coding in various computer programming languages.	3	3	2	3	3	3	2
3	Use the industry standard tools to analyze, design, develop and test software engineering based applications.	2	3	3	2	3	2	3
4	Apply theoretical knowledge to real-world engineering problems and manage complex engineering projects.	3	3	2	3	2	3	2
5	Acquire skills of collaboration and independent learning.	2	3	2	3	3	2	3

AHEPS:

C8	Identify and analyse ethical concerns and make reasoned ethical choices informed by professional codes of conduct
C9	Use a risk management process to identify, evaluate and mitigate risks (the effects of uncertainty) associated with a particular project or activity
C10	Adopt a holistic and proportionate approach to the mitigation of security risks
C12	Use practical laboratory and workshop skills to investigate complex problems
C15	Apply knowledge of engineering management principles, commercial context, project and change management, and relevant legal matters including intellectual property rights

C16	Function effectively as an individual, and as a member or leader of a team
C17	Communicate effectively on complex engineering matters with technical and non-technical audiences

ANNEXURE A EXPANDED MODULE SUMMARY

Table A.1 Expanded module-purpose summary

Module focus	Detailed academic significance
Identity and token handling	Anchors the legitimacy of every governed action by linking requests to accountable actors and session state.
Organization and membership logic	Provides the administrative context in which release rights and visibility become meaningful.
Application and version modeling	Transforms binaries into lifecycle-managed releases with durable identity and change context.
Audit and observability support	Preserves explainability of events and supports post-action review.
Recovery and backup procedures	Ensures that the platform remains defensible when normal operation is disrupted.

Annexure A extends the module discussion beyond the condensed descriptions given in the methodology chapter. The key insight is that each module contributes not only to functionality but also to academic interpretation. Identity logic supports the study of access legitimacy. Organization and membership logic support the study of scoped governance. Application and version modeling support the study of release lifecycle management. Audit and observability support the study of engineering accountability. Recovery procedures support the study of operational resilience.

When these modules are viewed together, the platform becomes a clear example of software engineering as systems thinking. The project does not revolve around a single algorithmic novelty. Instead, it demonstrates how multiple ordinary engineering concerns can be composed into a trustworthy release environment. This compositional strength is precisely what makes the project educationally valuable and worthy of detailed thesis treatment.

ANNEXURE B VERIFICATION MATRIX

Table B.1 Verification matrix for technical and operational checks

Verification area	Representative purpose	Interpretive value
Automated coverage	Exercise implementation paths and local correctness.	Supports trust in code-level discipline.
Direct smoke scenarios	Validate integrated workflows without routing-layer variation.	Supports trust in service interplay.
Gateway smoke scenarios	Validate routed user-facing paths and exposure boundaries.	Supports trust in ingress behavior and route discipline.
Documented API execution	Capture broader request/response evidence for real endpoints.	Supports trust in operationally visible behavior.
Operational scenario design	Specify resilience, load, and recovery evaluation pathways.	Supports trust in the seriousness of future readiness assessment.

The matrix in Annexure B reinforces that verification should not be collapsed into a single score. Different layers of evidence answer different questions. Automated tests help answer whether code paths behave as intended. Smoke checks help answer whether the integrated platform behaves coherently. Gateway validation helps answer whether exposure boundaries and routed interactions behave as expected. Operational scenario design helps answer whether the project understands what must be tested next to claim deeper readiness.

This distinction is especially important for a platform like VersionPark because release governance spans both application logic and operational handling. A system could appear correct under local testing yet still be poorly prepared for failure, weakly observable, or difficult to restore. The verification matrix therefore functions as an interpretive tool that shows why the thesis values multiple evidence types together rather than privileging only one.

ANNEXURE C GOVERNANCE AND RISK TREATMENT

Table C.1 Governance and risk treatment summary

Risk area	Generalized treatment approach
Unauthorized release visibility	Use authenticated and scoped access logic tied to organizational context.
Ambiguous release provenance	Preserve explicit application, version, and audit relationships.
Operational opacity	Use structured logs, trace identifiers, and audit recording.
State loss or corruption	Support verified backups, guarded restoration, and post-restore validation.

Documentation drift	Maintain architecture, validation, and recovery records as living technical evidence.
---------------------	---

Governance and risk treatment are central to the identity of the project. The system is designed to reduce specific organizational weaknesses: unclear release ownership, uncontrolled distribution scope, poor post-event traceability, and operational uncertainty during incidents. Even when the report abstracts away confidential details, these risk-treatment relationships remain visible and academically analyzable.

Annexure C also demonstrates that risk treatment is distributed across the platform. No single service or technology solves governance by itself. Authentication helps but is insufficient without scoped access logic. Object storage helps but is insufficient without version discipline. Recovery planning helps but is insufficient without integrity verification and operational validation. This distributed treatment of risk is a hallmark of mature systems engineering and one of the clearest lessons conveyed by the thesis.

ANNEXURE D ACADEMIC OUTCOME AND LEARNING SUMMARY

Table D.1 Extended academic outcome and learning summary

Learning dimension	What the project demonstrates
Problem analysis	The ability to reinterpret a familiar technical task as a broader governance and systems-engineering problem.
Design reasoning	The ability to justify modular boundaries, workflow decisions, and operational controls using explicit engineering logic.
Validation discipline	The ability to compare direct evidence, automated checks, and operational scenarios without overstating certainty.
Professional communication	The ability to explain realistic systems while preserving confidentiality and academic clarity.

Annexure D draws together the educational significance of the project. A thesis of this type is valuable not because it introduces a single novel algorithm, but because it shows how engineering judgment is exercised across multiple layers at once. The work required the interpretation of an organizational problem, the decomposition of that problem into bounded responsibilities, the design of explicit operational workflows, the integration of security and observability concerns, and the disciplined reporting of evidence. This breadth is an important indicator of final-year academic maturity.

The project also demonstrates that strong engineering outcomes often emerge from careful synthesis rather than from isolated innovation. VersionPark combines established tools and design ideas into a coherent answer to a realistic distribution challenge. The academic lesson is that originality in engineering can reside in the quality of integration, the clarity of governance, and the thoughtfulness of validation as much as in any single low-level technical novelty.

A further learning outcome concerns evidence management. The repository materials made it possible to differentiate between what is implemented, what is validated directly, what is planned for deeper evaluation, and what must remain generalized for confidentiality reasons. This distinction is professionally important. Engineers, researchers, and reviewers all benefit when claims are scaled to evidence and when unknowns are described honestly rather than concealed behind confident language.

Finally, the project shows the importance of communication discipline. The university template imposes a structure, but structure alone does not produce quality. Quality emerges when the writer can turn system design, operational reasoning, and result interpretation into a continuous argument. The present report therefore illustrates a broader academic achievement: the ability to convert a complex software system into a readable, inspectable, and defensible engineering narrative.

PROJECT DETAILS

<i>Student Details</i>			
Student Name	Diya Mohapatra		
Register Number	225805056	Section / Roll No	B/225805056
Email Address	diyamohapatra9@gmail.com	Phone No (M)	7894422003
<i>Project Details</i>			
Project Title	Development and Integration of Conversational AI Solutions		
Project Duration	6 months	Date of reporting	24 July 2026
<i>Organization Details</i>			
Organization Name	Avaamo AI		
Full postal address with pin code	20th Main Rd, Koramangala Industrial Layout, Koramangala, Bengaluru, Karnataka 560095		
Website address	https://avaamo.ai/		
<i>External Guide Details</i>			
Name of the Guide	Chetan KP		
Designation	Forward Deployment Engineer		
Full contact address with pin code	20th Main Rd, Koramangala Industrial Layout, Koramangala, Bengaluru, Karnataka 560095		
Email address	-	Phone No (M)	-
<i>Internal Guide Details</i>			
Faculty Name	Dr. Ashwitha A		
Full contact address with pin code	School of Computer Engineering, Manipal Institute of Technology, Bengaluru, Karnataka 560064, India.		
Email address	ashwitha.a@manipal.edu		

PLAGIARISM REPORT



Page 2 of 44 - Integrity Overview

Submission ID trn:oid:::1:3569688882

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **18 Not Cited or Quoted 2%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 1%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



Page 2 of 44 - Integrity Overview

Submission ID trn:oid:::1:3569688882